



Robotaksi Segment BT — Durum Raporu ve Mimari Rehber

Bu doküman sana **sıfırdan** anlatır: ağaç ne, neden segment, hangi dosya ne iş yapar, şu an neredeyiz, bundan sonra **senin** yapman gereken ne.

1. Behavior Tree (BT) Nedir?

Behavior Tree bir **karar ağacıdır**. Robot "şimdi ne yapayım?" sorusunu her saniye bu ağaca sorarak cevap alır.

Gerçek hayattan benzetme: **Sabah rutinin gibi düşün:**

Sabah Rutini (Sequence – sırayla yap):

- Alarm çaldı mı? (Condition)
- Kalk (Action)
- Kahvaltı yap (Action)
- İşe git (Action)

- Sequence (→)**: Çocuklarını **sırayla** çalıştırır. Biri başarısız olursa durur.
- Fallback (?)**: Çocuklarını sırayla dener, **ilk başarılı olan** yeter.
- Action**: Gerçek bir iş yapar (motoru çalıştır, dur, bekle)
- Condition**: Bir şeyi kontrol eder (yaya var mı? ışık kırmızı mı?)

Önemli: Ağaç her saniye **baştan** çalışır (tick). Bu sayede "yaya çıktı → dur" gibi refleksler anında devreye girer.

2. Neden "Segment" Yaklaşımı?

Eski yaklaşım (waypoint tabanlı):

Git → Nokta1 → Nokta2 → Nokta3 → ... → Park

Problem: Robot sadece "noktaya git" bilir. Kavşakta ne yapacağını, tünelde far açacağını, durakta bekleyeceğini **bilmez**.

Yeni yaklaşım (segment tabanlı):

Harita = graf (düğümler + kenarlar)

Düğümler: kavşak, durak, tünel girişi, park girişi...

Kenarlar (segmentler): iki düğüm arası yol parçası, HER BİRİNİN TİPİ VAR

Örnek rota:

START —LANE_FOLLOW—> KAVŞAK_A —INTERSECTION—> DURAK_1 —PASSENGER_STOP—> ...

Her segment tipinin FARKLI davranışı var:

Segment Tipi	Robot Ne Yapar	Şartname Puanı
--------------	----------------	----------------

LANE_FOLLOW	Düz şeritte sür	+30/nokta
INTERSECTION	Yavaşla, ışığa bak, tabelaya bak	+60/+40
ROUNDAABOUT	Dönel kavşakta boşluk bekle, gir, çık	—
TUNNEL	Far aç, yavaş geç, far kapat	+200
PASSENGER_STOP	Dur, 15-20sn bekle, yolcu al/bırak	+70/nokta
PARKING	Dik park yap (≤ 3 dk)	+80

3. Ağaç Yapısı — Groot'ta Gördüğün Şey

Groot'ta gördüğün ağaç **iç içe kutular** gibi düşün. En dıştan içe doğru:

3.1 MainTree (Groot'ta gördüğün ilk ekran)

MainTree (Sequence – sırayla yap):

- └─ SetBlackboard × 5 ← hız parametrelerini ayarla
- └─ LoadMission ← GeoJSON dosyasını oku, rota oluştur
- └─ DriveWithRecovery ← rotayı yürüt (hata olursa kurtarma dene)
- └─ FinishMission ← aracı durdur

Analoji: Sınava giriyorsun.

1. Kalemlerini hazırla (SetBlackboard)
2. Sınav kağıdını al (LoadMission)
3. Soruları çöz, takılırsan atla sonra dön (DriveWithRecovery)
4. Kağıdı teslim et (FinishMission)

3.2 DriveAllSegments (Expand'e tıklayınca)

DriveAllSegments:

- └─ Repeat (sonsuz döngü):
 - └─ SegmentLoop (Sequence):
 - └─ HasMoreSegments? ← daha segment var mı?
 - └─ GetCurrentSegment ← şu anki segmenti al (tip, hedef, meta)
 - └─ ExecuteSegment ← segmenti çalıştır (tip'e göre farklı davranış)
 - └─ AdvanceSegment ← sonraki segmente geç (index++)
 - └─ Inverter(HasMoreSegments) ← döngüden çıkış kontrolü

Analoji: Alışveriş listesi.

1. Listede daha ürün var mı? (HasMoreSegments)
2. Sıradaki ürünü oku: "süt" (GetCurrentSegment)
3. Süt reyonuna git ve al (ExecuteSegment)
4. Listede bir sonrakine geç (AdvanceSegment)
5. Liste bitene kadar tekrarla

3.3 ExecuteSegment (Switch/Case — tip'e göre yönlendir)

```
ExecuteSegment (Fallback — ilk eşleşen çalışır):
├ tip == LANE_FOLLOW?    → Seg_LaneFollow çalıştır
├ tip == INTERSECTION?   → Seg_Intersection çalıştır
├ tip == ROUNDABOUT?    → Seg_Roundabout çalıştır
├ tip == TUNNEL?         → Seg_Tunnel çalıştır
├ tip == PASSENGER_STOP? → Seg_PassengerStop çalıştır
├ tip == LANE_CHANGE?    → Seg_LaneChange çalıştır
└ tip == PARKING?        → Seg_Parking çalıştır
```

Analoji: Restoranda garson: "Müşteri ne istedi?" → Pizza → mutfağa gönder "Müşteri ne istedi?" → Çorba → çorbacıya gönder

3.4 SafeDrive (Her harekette çalışan güvenlik katmanı)

```
SafeDrive (ReactiveSequence — her tick ikisini de kontrol et):
├ SafetyReflexes ← tehlike var mı? (her saniye kontrol)
└ FollowLaneSegment ← şeritte sür (asıl hareket)
```

ReactiveSequence önemli: Normal Sequence sırayla çalışır ve biter. Ama ReactiveSequence **her tick** ilk çocuğu tekrar kontrol eder. Yani robot sürüyorken bile "yaya var mı?" sorusu **her saniye** sorulur.

Analoji: Araç kullanırken:

- Sol gözün yolda (FollowLane)
- Sağ gözün aynalarda (SafetyReflexes)
- Tehlike görürsen **anında** fren

3.5 SafetyReflexes (Güvenlik hiyerarşisi)

```
SafetyReflexes (ReactiveSequence):
├ 1. Acil Stop butonu basıldı mı? → DUR (sonsuz)
├ 2. Yaya var mı? → DUR, geçmesini bekle
├ 3. Dinamik engel var mı? → DUR, geçmesini bekle
└ 4. Statik engel var mı? → Şerit değiştir VEYA dur-bekle
```

Öncelik sırası kritik: Acil stop > Yaya > Dinamik > Statik. Yaya varken statik engele bakmaz bile.

Analoji: Yangın alarmı çalınca toplantıyı bırakırsın. Toplantıda telefon çalınca belki cevaplarsın. Ama yangın alarmı varken telefonu düşünmezsin.

3.6 Segment Alt-Ağaçları (Her biri farklı iş yapar)

Seg_LaneFollow (En basit — düz yol)

1. Hızı cruise_speed'e ayarla (0.60 m/s)
2. Görev noktası geçişini kaydet (+30 puan)
3. SafeDrive ile hedefe sür

Seg_Intersection (Kavşak — en karmaşık)

1. Hızı intersection_speed'e düşür (0.30 m/s)
2. Eğer Tur ≥ 2 ise:
 - a. Trafik ışığını kontrol et (kırmızı → dur, yeşile geç → kalk)
 - b. DUR tabelası var mı? (tam dur, sonra devam)
 - c. Dönüş yönü tabelayla çelişiyor mu? (evet → rotayı yeniden planla)
3. SafeDrive ile kavşağı geç
4. Hızı cruise'a geri al

Seg_Tunnel (Tünel)

1. Hızı tunnel_speed'e düşür
2. FAR AÇ 💡
3. SafeDrive ile tüneli geç
4. FAR KAPAT
5. Hızı cruise'a geri al

Seg_PassengerStop (Yolcu durağı)

1. SafeDrive ile durağa sür
2. Dur
3. Pozisyon kontrolü: 1m içinde mi?
 - Hayır → düzelt, tekrar dur, tekrar kontrol
4. 15-20 saniye bekle
5. Yolcu al/bırak sinyali gönder

Seg_Parking (Park)

1. Hızı parking_speed'e düşür
2. SafeDrive ile park girişine sür
3. Park girişi geçildi kaydet (+20 puan)
4. Uygun slot bul (P-3a tabelası olan, P-1 olmayan)
5. Dik park yap (≤ 3 dakika) (+80 puan)

4. Dosya Haritası — Ne Nerede?

```
robotaxi_bt/  
├── behaviour_trees/  
│   ├── segment_bt.xml          ← 🌲 ANA AĞAÇ (Groot'ta açtığın dosya)  
│   ├── segment_bt_groot_sema.md ← 📊 Ağaç şemaları (Mermaid diyagramları)  
│   ├── general_driving_bt.xml  ← 📦 Eski ağaç (artık kullanılmıyor)  
│   └── round1_bt.xml           ← 📦 Eski Tur 1 ağacı (artık kullanılmıyor)  
├── include/robotaxi_bt/  
│   ├── segment_bt_nodes.hpp    ← 📄 38 node'un C++ sınıf tanımları  
│   ├── segment_graph.hpp       ← 🗺️ Segment graf + Dijkstra rota planlama  
│   └── init_mission_action.hpp ← 📦 Eski InitMission (geriye dönük uyumluluk)  
└──
```

src/	
├─ segment_bt_nodes.cpp	← < 38 node'un STUB implementasyonları
├─ main.cpp	← 🚀 Test programı (tüm node'ları kayıt eder)
└─ init_mission_action.cpp	← 📦 Eski InitMission implementasyonu
scripts/	
└─ geojson_parser.py	← 🐍 GeoJSON → waypoints + segment eşleme
CMakeLists.txt	← 🛠 Build konfigürasyonu
package.xml	← 📦 ROS 2 paket tanımı

5. Şu An Neredeyiz?

✅ TAMAMLANAN İŞLER

#	İş	Durum
1	Ağaç mimarisi tasarlandı	✅ segment_bt.xml
2	Groot'ta görselleştirme	✅ Açılıyor, gezebiliyorsun
3	4 tasarım hatası düzeltildi	✅ Roundabout, PassengerStop, StaticObstacle, ValidateTurn
4	38 node'un C++ iskeleti yazıldı	✅ Derleniyor, 0 hata
5	Segment graf altyapısı	✅ SegmentGraph + Dijkstra
6	GeoJSON parser güncellendi	✅ --segments bayrağı
7	Build sistemi güncellendi	✅ colcon build geçiyor

❌ YAPILMASI GEREKEN İŞLER (SENİN TARAFIN)

#	İş	Ne demek?	Zorluk
1	FollowLaneSegment implementasyonu	Robot gerçekten şerit takip etsin	🔴 Zor
2	LoadMission implementasyonu	GeoJSON + graf → gerçek rota	🔴 Zor
3	Sensör condition node'ları	Yaya/engel/ışık algılama topic'lerini bağla	🟡 Orta
4	segment_map.yaml oluşturma	Harita ekibinden koordinat al, graf yaz	🟡 Orta
5	Actuator action node'ları	StopVehicle, TurnHeadlights, SetMaxSpeed	🟢 Kolay
6	Simülasyon testi	Gazebo'da Tur 1-2-3 senaryolarını dene	🟡 Orta

6. "Stub" Ne Demek? — En Önemli Kavram

Stub = sahte ama çalışan implementasyon.

Şu an tüm node'lar "çalışıyor" ama **gerçek bir iş yapmıyor**. Örnek:

```
// FollowLaneSegment – ŞU ANKI HALİ (stub):
BT::NodeStatus FollowLaneSegment::onRunning() {
    // TODO: Gerçek implementasyon
    // 2 saniye sonra "hedefe ulaştım" diyor (yalan söylüyor)
    if (elapsed > 2 saniye) return SUCCESS;
    return RUNNING;
}
```

```
// FollowLaneSegment – OLMASI GEREKEN HALİ (gerçek):
BT::NodeStatus FollowLaneSegment::onRunning() {
    // Kameradan şerit tespiti yap
    // PID ile direksiyon aç
    // Hedefe mesafeyi hesapla
    // Mesafe < 0.5m → SUCCESS
    // Mesafe > 0.5m → RUNNING (devam et)
    // Hata → FAILURE
}
```

Senin işin: Her node'daki `// TODO: Gerçek implementasyon` yorumunu silip, gerçek kodu yazmak.

7. Nereden Başlamalısın?

Adım 1: Ağacı anla (ŞİMDİ)

- Bu raporu oku 
- Groot'ta tüm SubTree'leri expand edip gez
- `segment_bt.xml` dosyasını oku — Türkçe yorumlar var

Adım 2: En kolay node'lardan başla

StopVehicle	→ cmd_vel'e sıfır hız gönder (3 satır kod)
SetMaxSpeed	→ parametreyi blackboard'a yaz (2 satır kod)
TurnHeadlights	→ GPIO pin toggle (5 satır kod)
HasMoreSegments	→ index < size karşılaştırması (ZATEN ÇALIŞIYOR)
IsSegmentType	→ string karşılaştırması (ZATEN ÇALIŞIYOR)

Adım 3: Sensör bağlantıları

PedestrianAhead	→ YOLO topic'ini subscribe et
DynamicObstacleAhead	→ Lidar clustering topic'ini subscribe et
TrafficLightAhead	→ Işık algılama topic'ini subscribe et

Adım 4: Asıl hareket (en zor)

FollowLaneSegment → Vision lane-following VEYA Nav2 FollowPath
LoadMission → GeoJSON parse + segment graf + rota

8. Pratik: Bir Node Nasıl İmplemente Edilir?

Örnek olarak `StopVehicle` 'ı gerçek hale getirelim:

Şu anki hali (`src/segment_bt_nodes.cpp`):

```
BT::NodeStatus StopVehicle::tick() {  
    RCLCPP_INFO(logger(), "StopVehicle: tam dur");  
    // TODO: Gerçek implementasyon  
    return BT::NodeStatus::SUCCESS;  
}
```

Gerçek hali olması gereken:

```
BT::NodeStatus StopVehicle::tick() {  
    // cmd_vel topic'ine sıfır hız gönder  
    auto msg = geometry_msgs::msg::Twist();  
    msg.linear.x = 0.0;  
    msg.angular.z = 0.0;  
    cmd_vel_pub_ -> publish(msg);  
  
    RCLCPP_INFO(logger(), "StopVehicle: tam dur - cmd_vel sıfırlandı");  
    return BT::NodeStatus::SUCCESS;  
}
```

Ama bunun için: Header'a `rclcpp::Publisher` eklemen ve constructor'da oluşturman lazım. Bunu yapmaya hazır olduğunda bana sor, birlikte yapalım.

9. Özet

Soru	Cevap
Ağaç ne?	Robotun karar mekanizması — "şimdi ne yapayım?"
Segment ne?	Yolun bir parçası, tipi var (düz yol, kavşak, tünel...)
XML ne işe yarıyor?	Ağacın yapısını tanımlıyor (Groot'ta gördüğün şey)
C++ ne işe yarıyor?	Her node'un gerçekte ne yaptığını tanımlıyor
Stub ne?	Sahte ama derlenebilir implementasyon (şu an bu var)
Ben ne yapacağım?	Stub'ları gerçek kodla dolduracaksın
En acil ne?	FollowLaneSegment + LoadMission
En kolay ne?	StopVehicle, SetMaxSpeed, TurnHeadlights